

Ghala Bin Salim S24109159

Tala Alzaeem S26101587

Sadeen Alshammari S24109624

Task 1:

| rain | hagrid | dumbledore | Knowledge Base true? |
|------|--------|------------|----------------------|
| F    | F      | F          | F                    |
| F    | F      | T          | F                    |
| F    | T      | F          | F                    |
| F    | T      | T          | F                    |
| T    | F      | F          | F                    |
| T    | F      | T          | TRUE                 |
| T    | T      | F          | F                    |
| T    | T      | T          | F                    |

The knowledge base is true only when `rain = True`, `hagrid = False`, and `dumbledore = True`. Therefore, in every model where the knowledge base is true, rain is also true. This is why `model_check(knowledge, rain)` prints **True**.

## 2. clue.py

MsScarlet: Mustard and Plum are ruled out, so MsScarlet must be the person.

Library: Kitchen and Ballroom are ruled out, so Library must be the room.

Knife: Revolver is ruled out. The unknown-card clue also rules out Wrench once MsScarlet and Library are established, so Knife must be the weapon. clue (1)

Task 2

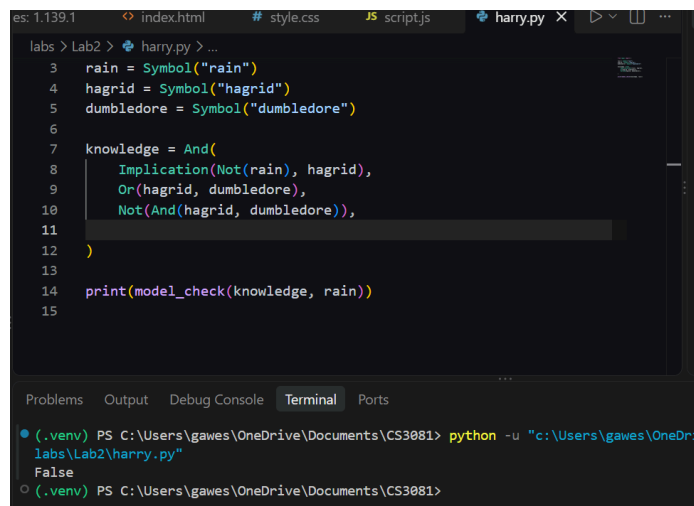
### 1. harry.py — Delete dumbledore

Prediction: False

Result: False

Explanation:

After removing the final dumbledore fact, rain is no longer guaranteed to be true in every model that satisfies the knowledge base. Therefore, the knowledge base does not entail rain, and the program prints False. The model checker only returns true when the query holds in every model satisfying the knowledge base. logic (1)



```
es: 1.139.1  index.html  # style.css  JS script.js  harry.py X  ...
labs > Lab2 > harry.py > ...
3  rain = Symbol("rain")
4  hagrid = Symbol("hagrid")
5  dumbledore = Symbol("dumbledore")
6
7  knowledge = And(
8      Implication(Not(rain), hagrid),
9      Or(hagrid, dumbledore),
10     Not(And(hagrid, dumbledore)),
11 )
12
13
14 print(model_check(knowledge, rain))
15

Problems  Output  Debug Console  Terminal  Ports
(.venv) PS C:\Users\gawes\OneDrive\Documents\CS3081> python -u "c:\Users\gawes\OneDrive\Documents\CS3081\Lab2\harry.py"
False
(.venv) PS C:\Users\gawes\OneDrive\Documents\CS3081>
```

## 2. puzzle.py — Delete MinervaGryffindor

Prediction: No specific person-house assignment will be guaranteed.

Result: The program prints nothing.

Explanation:

After deleting MinervaGryffindor, there are multiple possible arrangements that satisfy the remaining clues. Therefore, no individual person-house assignment is true in every valid model.

**New clue:**

```
knowledge.add( And( Symbol("GilderoyRavenclaw"), Symbol("PomonaHufflepuff"),
Symbol("MinervaGryffindor") ))
```

**Result after adding the new clue:**

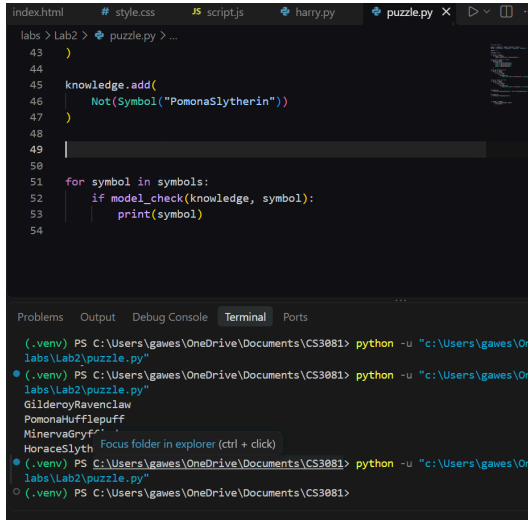
GilderoyRavenclaw

PomonaHufflepuff

MinervaGryffindor

HoraceSlytherin

**Explanation:** The new clue fixes Gilderoy, Pomona, and Minerva in different houses. Since each person can belong to only one house and each house can contain only one person, Horace must belong to Slytherin. Therefore, the arrangement becomes unique. puzzle (1)



```
index.html # style.css JS script.js harry.py puzzle.py X
labs > Lab2 > puzzle.py > ...
43 )
44
45 knowledge.add(
46     Not(Symbol("PomonaSlytherin"))
47 )
48
49
50
51 for symbol in symbols:
52     if model_check(knowledge, symbol):
53         print(symbol)
54
```

```
Problems Output Debug Console Terminal Ports
(.venv) PS C:\Users\gawes\OneDrive\Documents\CS3881> python -u "c:\Users\gawes\OneDrive\Documents\CS3881\puzzle.py"
(.venv) PS C:\Users\gawes\OneDrive\Documents\CS3881> python -u "c:\Users\gawes\OneDrive\Documents\CS3881\puzzle.py"
GilderoyRavenclaw
PomonaHufflepuff
MinervaGryff
HoraceSlyth
Focus folder in explorer (ctrl + click)
(.venv) PS C:\Users\gawes\OneDrive\Documents\CS3881> python -u "c:\Users\gawes\OneDrive\Documents\CS3881\puzzle.py"
(.venv) PS C:\Users\gawes\OneDrive\Documents\CS3881>
```

### Task 3

```
from logic import *

# Symbols
AKnight = Symbol("A is a Knight")
AKnave = Symbol("A is a Knave")
BKnight = Symbol("B is a Knight")
BKnave = Symbol("B is a Knave")

symbols = [AKnight, AKnave, BKnight, BKnave]

# Everyone is either a knight or a knave, but not both
basic_rules = And(
    Or(AKnight, AKnave),
    Not(And(AKnight, AKnave)),
    Or(BKnight, BKnave),
    Not(And(BKnight, BKnave))
)

def solve(name, knowledge):
    print(name)
```

```

    for symbol in symbols:
        if model_check(knowledge, symbol):
            print(symbol)
    print()

# ----- PUZZLE 0 -----
# A says: "I am both a knight and a knave."

statement0 = And(AKnight, AKnave)

knowledge0 = And(
    basic_rules,
    Implication(AKnight, statement0),
    Implication(AKnave, Not(statement0))
)

solve("Puzzle 0", knowledge0)

# ----- PUZZLE 1 -----
# A says: "We are both knaves."
# B says nothing.

statement1 = And(AKnave, BKnave)

knowledge1 = And(
    basic_rules,
    Implication(AKnight, statement1),
    Implication(AKnave, Not(statement1))
)

solve("Puzzle 1", knowledge1)

# ----- PUZZLE 2 -----
# A says: "We are the same kind."
# B says: "We are of different kinds."

same_kind = Or(

```

```

        And(AKnight, BKnight),
        And(AKnave, BKnave)
    )

different_kinds = Or(
    And(AKnight, BKnave),
    And(AKnave, BKnight)
)

knowledge2 = And(
    basic_rules,

    # A's statement
    Implication(AKnight, same_kind),
    Implication(AKnave, Not(same_kind)),

    # B's statement
    Implication(BKnight, different_kinds),
    Implication(BKnave, Not(different_kinds))
)

solve("Puzzle 2", knowledge2)

```

Output:

```

Problems  Output  Debug Console  Terminal  Ports
● (.venv) PS C:\Users\gawes\OneDrive\Documents\CS3081> python -u "c:\Users\gawes\OneDrive\Documents\CS3081\labs\Lab2\knights.py"
Puzzle 0
A is a Knave

Puzzle 1
A is a Knave
B is a Knight

Puzzle 2
A is a Knave
B is a Knight

○ (.venv) PS C:\Users\gawes\OneDrive\Documents\CS3081>

```

## Puzzle 0

**Result:** A is a Knave.

**Explanation:** A says, "I am both a knight and a knave." This statement cannot be true because every person must be either a knight or a knave, never both. Therefore, A is lying, so **A is a knave**.

## Puzzle 1

**Result:** A is a Knave and B is a Knight.

**Explanation:** A says, "We are both knaves." If A were a knight, the statement would have to be true, which would make A a knave and create a contradiction. Therefore, **A is a knave**. Since A's statement must be false, they cannot both be knaves, so **B must be a knight**.

## Puzzle 2

**Result:** A is a Knave and B is a Knight.

**Explanation:** A says they are the same kind, while B says they are different kinds. The consistent solution is **A is a knave and B is a knight**. A's statement is false because they are different kinds, and B's statement is true because they are different kinds.